

Court of Public Opinion

Technical Architecture Execution Checklist

****Purpose:**** Break the technical architecture into implementation phases that can be fed directly into an AI model.

****Suggested stack:**** Vite, TypeScript, Three.js, HTML/CSS overlay UI, Colyseus or equivalent authoritative room server, JSON-driven content

Phase 1. Frontend application shell

Goals

- Create the browser shell all features depend on.
- Separate rendering, UI, and gameplay state from the start.

Checklist

- Phase 1, Step 1: Create ``/src/app`` for app bootstrap and lifecycle.
- Phase 1, Step 2: Create ``/src/rendering`` for Three.js scene systems.
- Phase 1, Step 3: Create ``/src/ui`` for overlay components and HUD panels.
- Phase 1, Step 4: Create ``/src/game`` for rules, match state, and actions.
- Phase 1, Step 5: Create ``/src/data`` for loaders and schema validation.
- Phase 1, Step 6: Create ``/src/audio`` for music and one-shot sound hooks.
- Phase 1, Step 7: Create ``/src/debug`` for dev panels and inspectors.
- Phase 1, Step 8: Create an ``AppRoot`` that initializes renderer, scene, UI, and game state.
- Phase 1, Step 9: Add global error boundaries or fail-safe logging where practical.
- Phase 1, Step 10: Document folder ownership and file responsibilities.

Exit criteria

- Codebase structure is predictable.
- Systems have clear homes.

Phase 2. Three.js rendering architecture

Goals

- Build the 3D layer as a clean stage, not a tangled prototype.
- Support one main courtroom scene with fixed cameras.

Checklist

- Phase 2, Step 1: Create a renderer bootstrap module.
- Phase 2, Step 2: Create a scene bootstrap module.
- Phase 2, Step 3: Create a camera controller supporting fixed cinematic presets.
- Phase 2, Step 4: Add a main animation loop with delta time.
- Phase 2, Step 5: Add a resize-safe renderer and camera update path.
- Phase 2, Step 6: Add lighting suitable for PS1-style readability.
- Phase 2, Step 7: Add courtroom placeholder meshes first.
- Phase 2, Step 8: Add seat anchors for each role.
- Phase 2, Step 9: Add simple character placeholders or capsules before real models.
- Phase 2, Step 10: Add a scene state controller that reacts to trial phase and active speaker.

Exit criteria

- Courtroom scene renders reliably.
- Fixed camera flow is supported.
- Visual state can react to gameplay.

Phase 3. Asset pipeline

Goals

- Make models, textures, and UI assets load predictably.
- Keep style and filenames consistent.

Checklist

- Phase 3, Step 1: Choose MVP asset formats for models, textures, audio, and UI images.
- Phase 3, Step 2: Define import folders in `/public` or equivalent asset paths.
- Phase 3, Step 3: Add an asset manifest for courtroom props and character models.
- Phase 3, Step 4: Add loaders for models and textures.
- Phase 3, Step 5: Add fallback placeholder assets if loading fails.
- Phase 3, Step 6: Create a naming convention guide for all assets.
- Phase 3, Step 7: Add a preload path for critical assets.
- Phase 3, Step 8: Measure first-load impact and trim oversized files.
- Phase 3, Step 9: Add a visual asset validation scene for quick QA.
- Phase 3, Step 10: Lock export settings for artists and AI-generated asset cleanup.

Exit criteria

- Assets load consistently.
- The project is not blocked on missing final art.

Phase 4. Gameplay state architecture

Goals

- Keep gameplay rules deterministic and data-driven.
- Avoid logic being hidden in UI or scene code.

Checklist

- Phase 4, Step 1: Create a domain model for match state.
- Phase 4, Step 2: Create pure functions or equivalent handlers for gameplay actions.
- Phase 4, Step 3: Separate immutable state updates from visual side effects where practical.
- Phase 4, Step 4: Add a state event log for debugging.
- Phase 4, Step 5: Add selectors or query helpers for UI and AI logic.
- Phase 4, Step 6: Add serialization support for key match state if needed for multiplayer sync.
- Phase 4, Step 7: Add schema validation for incoming case data.
- Phase 4, Step 8: Add deterministic handling for timers, card play, and rulings.
- Phase 4, Step 9: Add unit tests for core reducers or action handlers.
- Phase 4, Step 10: Document state ownership boundaries.

Exit criteria

- The rules system is centralized.
- Match state is inspectable and testable.

Phase 5. Data schema and content loading

Goals

- Let predefined content drive the game.
- Make it easy for AI to generate new safe content later.

Checklist

- Phase 5, Step 1: Define schemas for cases, witnesses, evidence, jurors, cards, and role configs.
- Phase 5, Step 2: Store sample content in JSON or TypeScript data modules.
- Phase 5, Step 3: Add runtime validation for loaded content.
- Phase 5, Step 4: Add error output for malformed content.
- Phase 5, Step 5: Build a data loader layer separate from gameplay logic.
- Phase 5, Step 6: Add content IDs and reference integrity checks.

- Phase 5, Step 7: Add dev tooling to preview loaded content.
- Phase 5, Step 8: Add case linting rules for solvability and contradiction coverage.
- Phase 5, Step 9: Add sample packs for at least one complete trial.
- Phase 5, Step 10: Write content authoring docs for future AI prompts.

Exit criteria

- Content is structured.
- Bad data fails loudly.
- New cases can be added without code rewrites.

Phase 6. UI architecture

Goals

- Keep interface readable over the 3D scene.
- Support role-specific actions and phase-specific panels.

Checklist

- Phase 6, Step 1: Choose the UI framework or plain approach for overlays.
- Phase 6, Step 2: Create a root overlay layout.
- Phase 6, Step 3: Create reusable panel components.
- Phase 6, Step 4: Create role HUD components for Judge, Prosecutor, Defense, Witness, and Juror.
- Phase 6, Step 5: Create card list and card detail components.
- Phase 6, Step 6: Create evidence panel and evidence reveal screen.
- Phase 6, Step 7: Create transcript panel.
- Phase 6, Step 8: Create timer and active speaker indicators.
- Phase 6, Step 9: Create verdict and summary screens.
- Phase 6, Step 10: Add keyboard and mouse interaction support with clear focus rules.

Exit criteria

- UI panels are modular.
- Role actions are understandable.
- The scene and interface work together.

Phase 7. Audio and feedback systems

Goals

- Add punch and clarity to actions.

- Reinforce courtroom drama with minimal complexity.

Checklist

- Phase 7, Step 1: Create an audio manager interface.
- Phase 7, Step 2: Add support for background music and one-shot SFX.
- Phase 7, Step 3: Hook gavel, objection, evidence reveal, and verdict sounds into gameplay events.
- Phase 7, Step 4: Add screen shake or camera emphasis for key moments if appropriate.
- Phase 7, Step 5: Add portrait or character reaction states.
- Phase 7, Step 6: Add UI highlights for successful and failed actions.
- Phase 7, Step 7: Add optional subtitles or readable action captions.
- Phase 7, Step 8: Balance feedback so it remains funny but legible.
- Phase 7, Step 9: Add mute and volume controls.
- Phase 7, Step 10: Run a readability pass on all key event feedback.

Exit criteria

- Major actions have audiovisual feedback.
- The game feels alive without becoming noisy.

Phase 8. Multiplayer room architecture

Goals

- Support 2 to 6 players with authoritative state.
- Make low-player and solo modes work with AI seat fill.

Checklist

- Phase 8, Step 1: Choose Colyseus or equivalent room server approach.
- Phase 8, Step 2: Create a room state schema for connected players and assigned seats.
- Phase 8, Step 3: Create room join, leave, reconnect, and ready flows.
- Phase 8, Step 4: Assign roles and seats on match start.
- Phase 8, Step 5: Sync match state from server to clients.
- Phase 8, Step 6: Send player actions as validated commands, not direct state edits.
- Phase 8, Step 7: Add server-side validation for phase and role permissions.
- Phase 8, Step 8: Add reconnect-safe handling for in-progress matches.
- Phase 8, Step 9: Add AI seat fill when player count is low.
- Phase 8, Step 10: Test 2-player, 3-player, and 6-player lobby shapes.

Exit criteria

- Multiplayer is authoritative.

- Rooms are stable.
- Missing seats can be filled.

Phase 9. AI fallback architecture

Goals

- Use AI-like seat behavior without live dependency on generation.
- Keep bot logic deterministic enough for MVP.

Checklist

- Phase 9, Step 1: Create bot decision policies for Prosecutor, Defense, Judge, Witness, and Juror.
- Phase 9, Step 2: Base decisions on structured game state and content tags.
- Phase 9, Step 3: Add difficulty or personality presets for bots.
- Phase 9, Step 4: Add timeout-safe auto-play behavior.
- Phase 9, Step 5: Add logging so bot decisions can be inspected.
- Phase 9, Step 6: Ensure bots never access forbidden hidden information unless their role should know it.
- Phase 9, Step 7: Add support for solo mode against bot court cast.
- Phase 9, Step 8: Test bots across all shipped cases.
- Phase 9, Step 9: Tune bot pacing so matches do not stall.
- Phase 9, Step 10: Document all bot assumptions.

Exit criteria

- Empty seats can be filled.
- Solo play is possible.
- Bot behavior is inspectable.

Phase 10. Build, QA, and shipping readiness

Goals

- Produce a browser-ready MVP.
- Catch unstable systems before jam submission.

Checklist

- Phase 10, Step 1: Create production build scripts.
- Phase 10, Step 2: Run bundle size checks and trim oversized assets.
- Phase 10, Step 3: Add a smoke test checklist for local single-case playthrough.
- Phase 10, Step 4: Add a smoke test checklist for multiplayer room flow.

- Phase 10, Step 5: Test on desktop browser resolutions likely used by judges.
- Phase 10, Step 6: Check loading time and first interactive moment.
- Phase 10, Step 7: Remove obvious debug UI from release builds.
- Phase 10, Step 8: Add a crash and console error cleanup pass.
- Phase 10, Step 9: Build a release candidate and run a complete playthrough.
- Phase 10, Step 10: Freeze content and prepare jam submission package.

Exit criteria

- The build runs in browser.
- Core match flow is stable.
- Presentation is good enough for judging.

Architecture acceptance checklist

- Rendering, UI, game rules, and data loading are clearly separated.
- One courtroom scene is fully supported.
- Content is predefined and validated.
- Multiplayer uses authoritative commands.
- Bots can fill missing seats.
- The codebase can be handed back to Cursor in small phased chunks.